Mount Kenya University

PROJECT TITLE: SMART WARDROBE: A WEATHER-BASED CLOTHING RECOMMENDATION SYSTEM FOR KENYA

A project report submitted to the School of Computing and Informatics in partial fulfilment of the requirements for the award of the

DIPLOMA OF

INFORMATION TECHNOLOGY

At

MOUNT KENYA UNIVERSITY

Author:

WILLIAM MWIRIGIA (DIT/2023/59938)

OCTOBER 2026

DECLARATION

I declare that this project report entitled, **SMART WARDROBE: A WEATHER-BASED CLOTHING RECOMMENDATION SYSTEM FOR KENYA**, is my original work. It is submitted as a partial requirement for the award of the Diploma in Information Technology. It has never been researched anywhere else or presented in any institution of higher learning in the world for any academic award.

WILLIAM MWIRIGIA

ADM. No: DIT/2023/59938

Signature..... Date.....

SUPERVISOR

Name: Mr. Kevin Njagi

Signature..... Date.....

DEDICATION

This project is dedicated to the Almighty God who has given me life, good health and strength to pursue this course. I would also like to dedicate this work to my family and colleagues for having shown me love, patience and courage towards the completion of this project. May the Almighty God bless you and the work of your hands abundantly.

ACKNOWLEDGEMENT

I would like to acknowledge with sincere gratitude the Almighty God for the grace He granted me throughout this project. I thank Mount Kenya University for giving me the chance to pursue a Diploma in Information Technology. I wish to thank all my classmates and lecturers for contributing ideas and for providing a suitable working environment towards the completion of this work. I would like to thank Mr. Kevin Njagi for his valuable contribution, guidance and support in the development of this project.

ABSTRACT

Choosing what to wear is a daily decision that depends heavily on the weather, yet most people in Kenya must check a weather application, interpret the figures themselves and then decide on suitable clothing. This process is not personalised, ignores differences between men, women and children, and offers little guidance for formal and casual occasions. This project presents **Smart Wardrobe**, a web-based system that combines live weather data with artificial intelligence to give clear clothing recommendations for Kenyan conditions.

The system was built with PHP, MySQL, HTML, CSS, Bootstrap and JavaScript. It retrieves current weather data (temperature, "feels like" temperature, humidity, wind speed and conditions) for Kenyan cities from the OpenWeatherMap service, passes the readings to a large language model through the OpenRouter service, and displays separate recommendations for men, women and children. Users can also chat with a Fashion Advisor, view outfit images, search the weather of any city and manage their profile. An administrator panel allows user accounts to be added, edited and deleted, supports a secure password reset using a one-time password sent by SMS, and produces a users report as an image.

The project followed an Agile development approach in four sprints. This report documents the problem, objectives, feasibility, literature, methodology, analysis and design models, implementation, test plan, limitations and recommendations for the system.

Contents

DECLARATION **2**
DEDICATION **3**
ACKNOWLEDGEMENT **3**
ABSTRACT **4**
CHAPTER 1: INTRODUCTION **7**
1.0 Introduction 7
1.1 Background of the Project 7
1.2 Problem Statement 8
1.3 Proposed Solution 8
1.4 Objectives of the Project 9
1.5 Justification of the Project 9
1.6 Scope of the Project 9
1.7 Budget for the Smart Wardrobe System 10
1.8 Time Schedule for the Smart Wardrobe Project 11
1.9 Gantt Chart 12
1.10 Feasibility Study of the Smart Wardrobe System 12
1. Technical Feasibility 12
2. Operational Feasibility 12
3. Economic Feasibility 13
4. Legal and Ethical Feasibility 13
CHAPTER 2: LITERATURE REVIEW **14**
2.0 Introduction 14
2.1 Weather and Clothing Comfort 14
2.2 Recommender Systems and Fashion 14
2.3 Weather Data Services 14
2.4 Artificial Intelligence for Text and Images 15
2.5 Web Security and Data Protection 15
2.6 Existing Solutions and the Gap 15
2.7 Conclusion 16
CHAPTER 3: METHODOLOGY **17**
3.0 Introduction 17
3.1 Research Design 17
3.2 System Development Model 17
1. Concept and Planning 17
2. Product Backlog 17
3. Sprint Planning and Execution 17
4. Daily Check-ins, Review and Retrospective 18
5. Final Evaluation and Release 18
3.3 Data Collection 18
3.4 Target Population and Sampling Techniques 18
3.5 Data Analysis 18
3.6 Tools and Technologies 18
3.7 Limitations 19
CHAPTER 4: SYSTEM ANALYSIS, DESIGN AND IMPLEMENTATION **20**
4.1 System Analysis 20
4.1.1 Existing System 20

4.1.2 Proposed System 20
4.1.3 Functional Requirements 20
4.1.4 Non-Functional Requirements 21
4.1.5 System Requirements 21
4.2 Requirements Modelling 22
a. Context Diagram 22
b. Data Flow Diagram (DFD) 23
c. Entity Relationship Diagram (ERD) 23
d. Flowchart Diagram 24
4.3 System Design 25
4.3.1 System Architecture 25
4.3.2 Database Design 25
4.3.3 Module Design 26
4.3.4 Interface Design 26
4.4 Implementation 27
4.4.1 Development Environment 27
4.4.2 Authentication Module 27
4.4.3 Weather Module 27
4.4.4 AI Recommendation Module 27
4.4.5 Fashion Advisor and Outfit Images 28
4.4.6 Profile Module 28
4.4.7 Administrator Module 28
4.4.8 SMS One-Time Password Reset (Administrator) 28
4.5 System Testing 29
4.6 Installation and User Guide 30
4.6.1 Installation Steps 30
4.6.2 User Guide 30
CHAPTER 5: LIMITATIONS, RECOMMENDATIONS AND CONCLUSION 32
5.0 Challenges and Limitations of the Current Build 32
5.1 Recommendations 33
5.2 Future Work 33
5.3 Conclusion 34
5.4 References 34
APPENDIX A: SOURCE FILE INVENTORY 35
APPENDIX B: DATABASE SCRIPT 37

CHAPTER 1: INTRODUCTION

1.0 Introduction

Weather affects almost every daily decision, and clothing is one of the most immediate. A person who dresses lightly for a cold, wet morning in Nakuru or who wears heavy layers for a humid afternoon in Mombasa quickly feels the cost of a poor decision. Although weather information is freely available on phones and websites, it is presented as raw figures such as temperature, humidity and wind speed. The user must still translate these figures into a decision about what to wear.

This project proposes **Smart Wardrobe**, a web-based Weather-Based Clothing Recommendation System. The system reads real-time weather data for Kenyan cities and uses artificial intelligence to translate it into practical clothing advice for men, women and children, for both casual and formal occasions. It also provides a conversational Fashion Advisor, outfit image suggestions, user accounts, and an administrator panel for managing the system.

Key Features:

- **Live Weather Data**: Current temperature, "feels like" temperature, humidity, wind speed and conditions for Kenyan cities and any searched city.
- **AI Clothing Recommendations**: Advice for men, women and children, covering casual and formal wear, layering and protection from sun and rain.
- **Fashion Advisor Chatbot**: A chat interface where users ask fashion questions and receive instant answers.
- **Outfit Image Gallery**: Visual outfit ideas matched to the recommendation categories.
- **User Accounts and Profiles**: Signup, login, logout, password reset and profile updates.
- **Administrator Panel**: User management (add, edit, delete), weather analysis with charts, and a downloadable users report.

Expected Benefits:

- Faster and more confident daily clothing decisions.
- Advice that is relevant to Kenyan weather conditions and to different age and gender groups.
- A single place for weather information and clothing guidance.
- Simple administration of users through a secure dashboard.

1.1 Background of the Project

Kenya has a varied climate. The coast is hot and humid, the highlands around Nairobi, Nakuru and Meru are cool and often wet, while the western and northern regions can be very hot. Weather can also change quickly within a single day. As a result, residents, students and travellers frequently need to adjust what they wear.

Weather forecasting services have become widely accessible through application programming interfaces (APIs). At the same time, advances in artificial intelligence, and in particular large language models, make it possible to turn numerical data into natural-language advice. Combining these two technologies allows a system to explain, in plain words, what a person should wear for the conditions outside.

Smart Wardrobe builds on this opportunity. Instead of leaving users to interpret weather figures, the system collects the data, builds a structured request for an AI model, and presents the response in an organised format that is easy to read on a phone or computer.

1.2 Problem Statement

Most people decide what to wear by looking out of the window or by reading a temperature on a weather application. These sources give numbers but not guidance. They do not say whether a light jacket is enough, whether an umbrella is needed, or what is suitable for a child compared with an adult. Generic weather applications are also not tailored to Kenyan cities and local conditions.

Wrong clothing choices lead to discomfort, illness from exposure to cold or rain, sunburn, and wasted time changing clothes. Users who want fashion advice must search separately on social media or websites, and the advice found is rarely linked to the actual weather. There is therefore a need for a single, simple system that links live weather data to clear clothing recommendations for different groups of people.

1.3 Proposed Solution

The proposed solution is **Smart Wardrobe**, a web application that automatically converts weather data into clothing advice. The system has the following features:

1. **Real-Time Weather Retrieval**: The system calls the OpenWeatherMap service to obtain current conditions for Kenyan cities or any city entered by the user.
2. **AI-Generated Recommendations**: The weather readings are placed in a structured prompt and sent to an AI model, which returns clothing advice for men, women and children, including casual and formal options.
3. **Interactive Fashion Advisor**: A chat interface allows users to ask follow-up fashion questions.
4. **Outfit Visualisation**: Outfit images are displayed to help users picture the recommendations.
5. **Secure Access**: Users register and log in before using the system, and session control restricts pages to signed-in users.
6. **Administrator Dashboard**: Administrators manage users, view weather analysis charts, and generate a users report.

7. **Responsive Interface**: The interface is built with Bootstrap and CSS so that it works on phones, tablets and computers.

1.4 Objectives of the Project

Main Objective

To design and develop a web-based system that recommends suitable clothing to users based on real-time weather conditions in Kenya.

Specific Objectives

1. To retrieve and display accurate, real-time weather data for Kenyan cities and for any city searched by the user.
2. To generate clothing recommendations for men, women and children using artificial intelligence.
3. To provide an interactive Fashion Advisor chatbot and outfit image suggestions.
4. To implement user registration, authentication, session management and profile updates.
5. To provide an administrator panel for managing users and producing reports.
6. To implement a secure password reset process using one-time passwords sent by SMS.
7. To test the system and document its design, implementation and limitations.

1.5 Justification of the Project

1. **Daily Relevance**: Clothing decisions are made every day, so even small improvements in guidance benefit users repeatedly.
2. **Health and Comfort**: Correct clothing reduces exposure to cold, rain and strong sun.
3. **Local Context**: The system focuses on Kenyan cities and conditions rather than a generic global audience.
4. **Use of Modern Technology**: The project applies web development, API integration and artificial intelligence, which are in demand skills in the technology sector.
5. **Low Cost**: The system relies on free or low-cost services and open-source tools such as PHP and MySQL, making it affordable to build and host.
6. **Accessibility**: Being web-based, it works on any device with a browser and an internet connection and needs no installation.
7. **Foundation for Growth**: The modular design allows future additions such as a personal digital wardrobe, event planning and mobile applications.

1.6 Scope of the Project

The project covers the design, development and testing of a web application with the following scope:

1. **Weather Data**: Current weather for Kenyan cities and for any city typed into the search box. The weather service is configured with a list of eleven major cities (Nairobi, Mombasa, Kisumu, Nakuru, Eldoret, Thika, Malindi, Kitale, Kakamega, Kisii and Meru); in the current build Nairobi is the default city used for recommendations to keep response times short.
2. **Recommendations**: AI-generated advice for men, women and children, for casual and formal occasions.
3. **User Module**: Signup, login, logout, password reset and profile management (email and password changes).
4. **Fashion Advisor**: A chat interface connected to an AI model, and an outfit image gallery.
5. **Administrator Module**: Admin login, user listing, add, edit and delete user, weather analysis with charts, profile management, SMS-based password reset and a PNG users report.
6. **Resources**: A blog and resources page with fashion tips, sustainability guides and event guides.

The project does **not** include a mobile application, online shopping or payment, automatic recognition of clothes from photographs, or long-range (multi-day) forecasting. These are listed as future work.

1.7 Budget for the Smart Wardrobe System

The table below shows the estimated cost of developing and running the system for one year. Because the project uses open-source tools and free-tier services, the cost is mainly hosting, domain registration, connectivity and printing. Amounts are estimates in Kenya Shillings (KES) and may vary with the provider chosen.

Item	Quantity	Unit Cost (KES)	Total Cost (KES)
Development computer (already owned)	1	0	0
Domain name (.co.ke), 1 year	1	1,200	1,200
Shared web hosting (PHP and MySQL), 1 year	1	6,000	6,000
SSL certificate (Let's Encrypt)	1	0	0
OpenWeatherMap API (free tier)	1	0	0
OpenRouter AI API (free models; optional credit top-up)	1	2,000	2,000
SMS credits for one-time passwords	1	1,000	1,000
Internet bundles (3 months)	3	1,500	4,500
Printing and binding of documentation	1	1,500	1,500
Contingency and testing	1	3,000	3,000

Item	Quantity	Unit Cost (KES)	Total Cost (KES)
TOTAL ESTIMATED BUDGET			**19,200**

Explanation: The software stack (PHP, MySQL, Apache, Bootstrap, Chart.js and Composer libraries) is open source and carries no licence fee. Hosting and the domain are the main recurring costs. The AI service can run on free models during development, while a small credit top-up is allowed for in case higher limits are needed during testing and demonstration.

1.8 Time Schedule for the Smart Wardrobe Project

The project is planned to take **12 weeks**. The following table shows the activities, their durations and their start and end weeks.

Phase	Activity	Duration	Start	End
Phase 1: Planning and Design	Requirements gathering and feasibility study	2 weeks	Week 1	Week 2
	System design (database, interface, diagrams)	2 weeks	Week 2	Week 3
Phase 2: Development	Database, signup, login and sessions	2 weeks	Week 3	Week 4
	Weather module (OpenWeatherMap)	1 week	Week 5	Week 5
	AI recommendations, chatbot and outfit images	2 weeks	Week 6	Week 7
	Administrator panel and reports	2 weeks	Week 7	Week 8
Phase 3: Testing and Integration	Integration of modules	3 weeks	Week 7	Week 9
	System testing and bug fixing	2 weeks	Week 9	Week 10
Phase 4: Implementation	Deployment, user training and handover	1 week	Week 11	Week 11
Phase 5: Documentation and Evaluation	Project documentation	4 weeks	Week 9	Week 12
	Evaluation, maintenance plan and final report	2 weeks	Week 11	Week 12

Total Duration: 12 Weeks

1.9 Gantt Chart

The Gantt chart below shows how the main tasks are spread across the 12 weeks. Shaded cells show the weeks in which each task is active.

Figure 1.1: Project Gantt chart (12 weeks)

Task / Week	1	2	3	4	5	6	7	8	9	10	11	12
Planning												
Design												
Development												
Integration												
Testing												
Implementation												
Documentation												
Evaluation												
Maintenance												

1.10 Feasibility Study of the Smart Wardrobe System

The feasibility study assesses whether the project is practical in four areas: technical, operational, economic, and legal and ethical.

1. Technical Feasibility

- **Hardware**: The system needs only a standard computer for development and a web server for hosting. Users need any device with a browser, such as a phone, tablet or laptop.
- **Software**: PHP, MySQL, Apache, HTML, CSS, JavaScript, Bootstrap and Chart.js are mature, well documented and free. Composer manages the PHP libraries (Guzzle and PHPMailer are declared in the project).
- **External Services**: OpenWeatherMap and OpenRouter provide documented REST APIs that return JSON, which PHP can consume with cURL.
- **Skills**: The developer has the web development skills acquired during the diploma programme, so the technology is within reach.

2. Operational Feasibility

- **Ease of Use**: The interface is simple. A user signs in, picks or searches a city, and reads the advice.
- **Training**: Little training is needed for users. Administrators only need to learn the dashboard, which has clearly labelled Add, Edit and Delete actions.
- **Maintenance**: Maintenance involves keeping API keys valid, backing up the database and updating PHP libraries.

- **Acceptance**: Because weather and clothing are part of everyday life, users are likely to adopt the system readily.

3. Economic Feasibility

- **Initial Costs**: Estimated at KES 19,200 in total, mostly for hosting, domain and connectivity (see Section 1.7).
- **Running Costs**: Hosting and domain renewals are the main recurring costs. The free tiers of the weather and AI services are sufficient for a small user base.
- **Benefits**: The system saves users time, improves comfort and offers a platform that could later be monetised through partnerships or advertising.

4. Legal and Ethical Feasibility

- **Data Protection**: The system stores usernames, email addresses and passwords. Under the Kenya Data Protection Act, 2019, this personal data must be collected for a stated purpose, kept secure and not shared without consent.
- **API Terms**: Use of OpenWeatherMap and OpenRouter must follow their terms of service and rate limits.
- **Responsible AI**: AI output can be inaccurate or biased. Advice is therefore presented as guidance, and users keep the final decision.
- **Intellectual Property**: Images and generated content should only be used where licences allow.

CHAPTER 2: LITERATURE REVIEW

2.0 Introduction

This chapter reviews published work and technologies related to weather-based clothing recommendation. It covers the link between weather and clothing comfort, recommender systems, weather data services, artificial intelligence for text and image generation, and web security and data protection. It ends by identifying the gap that Smart Wardrobe aims to fill.

2.1 Weather and Clothing Comfort

The relationship between the environment and human comfort has been studied for decades. Fanger (1970) showed that thermal comfort depends on air temperature, humidity, air movement, radiant heat, activity level and the insulation provided by clothing. Industry standards such as ASHRAE Standard 55 build on this work to define conditions that most people find acceptable.

Two points from this research are important for Smart Wardrobe. First, temperature alone is not enough. Wind and humidity change how warm or cold a person feels, which is why weather services publish a "feels like" temperature. Second, clothing is a practical way to adjust personal comfort. A system that recommends clothing must therefore use several weather variables and not only the temperature reading.

2.2 Recommender Systems and Fashion

Recommender systems suggest items to users based on data about the users, the items or the context. Ricci, Rokach and Shapira (2015) describe the main approaches, including content-based, collaborative and context-aware methods. Context-aware recommenders use information such as time, location and weather to make suggestions more relevant.

Fashion is a common application area for recommenders, because personal taste, occasion and season all influence what a person wears. Most fashion recommenders concentrate on products to buy and use browsing or purchase history. Fewer systems use live weather as the main context, and fewer still are designed for African climates and clothing habits. Smart Wardrobe treats weather as the primary context and Kenyan conditions as the target environment.

2.3 Weather Data Services

Modern weather services expose their data through web APIs. A client application sends an HTTP request containing a location and an API key, and receives a structured JSON response with the current conditions. OpenWeatherMap is one such service. Its current weather endpoint returns temperature, "feels like" temperature, humidity, pressure, wind speed, visibility, a short description of the conditions and an icon code, and it supports metric units (OpenWeather, n.d.).

Using an external service avoids the cost of owning weather sensors or running forecasting models. It also introduces dependencies: the system needs an internet connection, must respect rate limits, and must protect the API key. Good practice is to handle errors gracefully, set request time-outs and store keys outside publicly accessible files.

2.4 Artificial Intelligence for Text and Images

Large language models are neural networks trained on very large text collections. They are based on the Transformer architecture introduced by Vaswani et al. (2017) and have been shown to perform many language tasks from only a short instruction, or prompt (Brown et al., 2020). This makes it possible to give a model weather readings and ask for clothing advice in natural language, without training a custom model.

Services such as OpenRouter provide a single API through which many language models can be called, including models that are free to use within limits (OpenRouter, n.d.). For image creation, diffusion models such as those described by Rombach et al. (2022) generate pictures from text descriptions. In this project, an experimental image generation script was tested, while the production interface displays curated outfit images matched to each recommendation category.

A known limitation of language models is that their answers can be inaccurate, inconsistent or biased. For this reason a recommender such as Smart Wardrobe should present output as guidance, provide fallback content when the AI service fails, and structure the prompt carefully so that responses follow the expected format.

2.5 Web Security and Data Protection

Web applications that handle user accounts face common risks. The OWASP Top 10 (OWASP Foundation, 2021) lists broken access control, injection and cryptographic failures among the most serious. Recommended defences include prepared statements for database queries, hashing passwords with a strong one-way algorithm such as the one provided by PHP's password_hash function, validating and escaping user input and output, and keeping secrets out of source files.

In Kenya, the Data Protection Act, 2019 requires organisations that process personal data to do so lawfully, for a specified purpose, with appropriate security safeguards. Even a small student system that stores names and email addresses should follow these principles.

2.6 Existing Solutions and the Gap

General weather applications show temperature and forecasts, and some show a brief outfit tip. Fashion websites and social media offer style ideas, but these are not tied to the weather at the user's location. Neither category typically provides, in one place, separate guidance for men, women and children, local city weather, an interactive advisor and an administrator panel.

Aspect	Typical weather application	Smart Wardrobe
Main output	Numbers and icons	Plain-language clothing advice
Audience	General	Men, women and children
Occasion	Not covered	Casual and formal
Interaction	Search only	Search plus Fashion Advisor chat
Local focus	Global	Kenyan cities and conditions
Administration	None	User management and reports

2.7 Conclusion

The literature shows that comfort depends on several weather variables, that context-aware recommenders are an established approach, that weather APIs and language models are easy to integrate, and that security and data protection must be built in from the start. Smart Wardrobe combines these ideas into a single system focused on Kenyan conditions, which addresses the gap identified above.

CHAPTER 3: METHODOLOGY

3.0 Introduction

This chapter explains the approach used to plan, design, build and evaluate Smart Wardrobe. It describes the research design, the development model, how data was collected and analysed, the tools used, and the limitations of the study.

3.1 Research Design

The project used an applied, developmental research design. It identifies a practical problem, builds a working system to solve it, and then tests the system to see whether it meets its objectives. A mixed approach was planned: quantitative measures such as response time and the number of test cases passed, and qualitative feedback from users about clarity and usefulness of the advice.

3.2 System Development Model

The **Agile** model was chosen because requirements became clearer as prototypes were reviewed and because working software can be delivered in small increments (Schwaber & Sutherland, 2020). The project was divided into four sprints of about three weeks each.

1. Concept and Planning

- **Objective**: Define the problem, goals, scope and timeline (12 weeks).
- **Activities**: Study existing weather and fashion applications, choose the technology stack, and agree on the main features.

2. Product Backlog

The backlog listed all features: user signup and login, weather retrieval, weather search, AI recommendations, Fashion Advisor chat, outfit images, profile management, administrator panel, password reset by SMS, reports, and testing.

3. Sprint Planning and Execution

Sprint	Weeks	Main work	Deliverable
Sprint 1	1 to 3	Requirements, design, database creation, signup, login, logout and session control	Working user accounts
Sprint 2	4 to 6	Weather module using OpenWeatherMap, weather search page, weather icons and layout	Live weather display
Sprint 3	7 to 9	AI prompt builder, clothing recommendations page, Fashion Advisor chat, outfit images, profile page	AI advice and chat
Sprint 4	10 to 12	Administrator panel, SMS password reset, reports, testing, deployment and documentation	Complete system

4. Daily Check-ins, Review and Retrospective

- **Daily check-in**: A short self-review of what was done, what is next and any blockers.
- **Sprint review**: Demonstrate the working features to the supervisor and collect feedback.
- **Retrospective**: Note what worked and what to improve before the next sprint.

5. Final Evaluation and Release

After the fourth sprint the system was tested end to end, deployed on a web server, and handed over together with this documentation.

3.3 Data Collection

- **Requirements data**: Gathered through observation of how people check the weather and decide on clothing, informal interviews with classmates and lecturers, and a short questionnaire.
- **Weather data**: Collected automatically from the OpenWeatherMap API at the time a user opens the weather page or runs a search.
- **AI responses**: Collected from the OpenRouter API for each recommendation request.
- **User account data**: Username, email and password collected during signup and stored in the MySQL database.

3.4 Target Population and Sampling Techniques

The target population is people in Kenya who use phones or computers daily and make clothing decisions based on weather, such as students, lecturers, office workers and parents. A sample of about 30 respondents drawn from Mount Kenya University students and staff is planned for the evaluation stage.

- **Purposive sampling**: Selecting respondents who regularly check weather information.
- **Convenience sampling**: Using people who are easily reachable on campus, given the limited time of a diploma project.

3.5 Data Analysis

- **Quantitative**: Test case results will be counted as passed or failed, response times noted, and questionnaire scores summarised using Microsoft Excel (percentages, averages and charts).
- **Qualitative**: Open comments on clarity, usefulness and design will be grouped into themes and used to propose improvements.

3.6 Tools and Technologies

Tool / Technology	Purpose in the project
PHP 8	Server-side programming for authentication, weather, AI and admin modules
MySQL (wardrobe_db)	Database for user and administrator accounts
Apache (XAMPP)	Local web server for development and testing
HTML5, CSS3, JavaScript	Page structure, styling and interaction
Bootstrap 5 and Font Awesome	Responsive layout and icons
Chart.js	Line and pie charts in the weather analysis page
cURL / Guzzle (Composer)	Calling external APIs
OpenWeatherMap API	Real-time weather data
OpenRouter API	Access to the language model that produces advice and chat replies
PHP GD library	Drawing the PNG users report
SMS gateway	Sending one-time passwords to administrators
Visual Studio Code, browser developer tools	Coding, debugging and testing

3.7 Limitations

- **Dependence on external services**: If OpenWeatherMap or OpenRouter is slow or unavailable, weather or advice may not load. Free AI models have usage limits.
- **Quality of AI output**: Responses can vary in wording and quality and may need fallback text.
- **Limited sample**: The evaluation sample is small and drawn from one institution, so results may not represent all Kenyans.
- **Time and budget**: The 12-week period and student budget limited advanced features such as a mobile app and multi-day forecasts.

CHAPTER 4: SYSTEM ANALYSIS, DESIGN AND IMPLEMENTATION

4.1 System Analysis

4.1.1 Existing System

At present a person who wants to know what to wear opens a weather application or website, reads the temperature and conditions, and decides alone. If advice on style is needed, separate searches are made on social media or fashion sites. The process is manual, spread across several sources, and not linked to the user's city or to the occasion.

4.1.2 Proposed System

The proposed system gathers live weather data, asks an AI model to interpret it, and presents the advice for men, women and children in one interface. It adds user accounts, a Fashion Advisor chat, outfit images, and an administrator panel.

4.1.3 Functional Requirements

ID	Module	Requirement
FR1	Authentication	The system shall allow a visitor to register with a username, email and password, rejecting empty fields, invalid emails and duplicate usernames or emails.
FR2	Authentication	The system shall allow a registered user to log in and shall restrict weather, clothing and profile pages to signed-in users.
FR3	Authentication	The system shall allow a user to log out, ending the session.
FR4	Weather	The system shall retrieve current weather (temperature, feels-like, humidity, wind speed, conditions) for Kenyan cities from OpenWeatherMap.
FR5	Weather	The system shall allow a user to search the weather of any city and shall display an error message if the city is not found.
FR6	Recommendations	The system shall build a prompt from the weather data and obtain clothing recommendations for men, women and children from an AI model.
FR7	Recommendations	The system shall display the recommendations by category together with the time of the last update, and shall show a fallback message if the AI service fails.
FR8	Fashion Advisor	The system shall provide a chat interface in which users send questions and receive replies from an AI model.
FR9	Outfit images	The system shall display outfit images for male, female and children categories and allow an image to be viewed in detail and downloaded.
FR10	Profile	The system shall allow a user to change their email and password after confirming the current password.
FR11	Admin	The system shall allow an administrator to log in, view all users, and add, edit and delete users.

ID	Module	Requirement
FR12	Admin	The system shall provide a weather analysis page with line and pie charts and a print option.
FR13	Admin	The system shall allow an administrator to update their profile, including a phone number, and reset a forgotten password using a one-time password sent by SMS.
FR14	Reports	The system shall generate a users report (ID, username, email, role and join date) as a PNG image for administrators only.
FR15	Resources	The system shall provide a blog and resources page with fashion tips, sustainability guides and event guides.

4.1.4 Non-Functional Requirements

- **Usability**: Clear navigation, icons, readable cards and a layout that adapts to phone, tablet and desktop screens.
- **Performance**: Pages should load quickly. External requests use time-outs (10 seconds for weather and 30 seconds for AI) so that the page does not hang.
- **Reliability**: When the weather or AI service fails, the system should display an understandable message or sample recommendations.
- **Security**: Sessions protect private pages; database queries should use prepared statements; passwords should be hashed; API keys should be kept out of public files.
- **Maintainability**: The code is separated into modules (authentication, weather, AI, admin) so that each can be changed independently.
- **Portability**: The system runs on any server with PHP and MySQL, on Windows, Linux or macOS.

4.1.5 System Requirements

Item	Minimum	Recommended
Server processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz or higher
Server memory	2 GB RAM	4 GB RAM or more
Storage	5 GB free	20 GB free (for growth and backups)
Operating system	Windows 10, Linux or macOS	Linux server (Ubuntu)
Web server	Apache 2.4 (XAMPP)	Apache 2.4 with HTTPS
PHP	PHP 8.0 with cURL, mysqli and GD	PHP 8.4 (as declared in composer.json)
Database	MySQL 5.7 or MariaDB 10	MySQL 8
Client device	Any modern browser, internet connection	Chrome, Edge, Firefox or Safari, latest version

4.2 Requirements Modelling

a. Context Diagram

The context diagram shows Smart Wardrobe as a single process and the external entities that exchange data with it: the user, the administrator, the OpenWeatherMap service, the OpenRouter AI service and the SMS gateway.

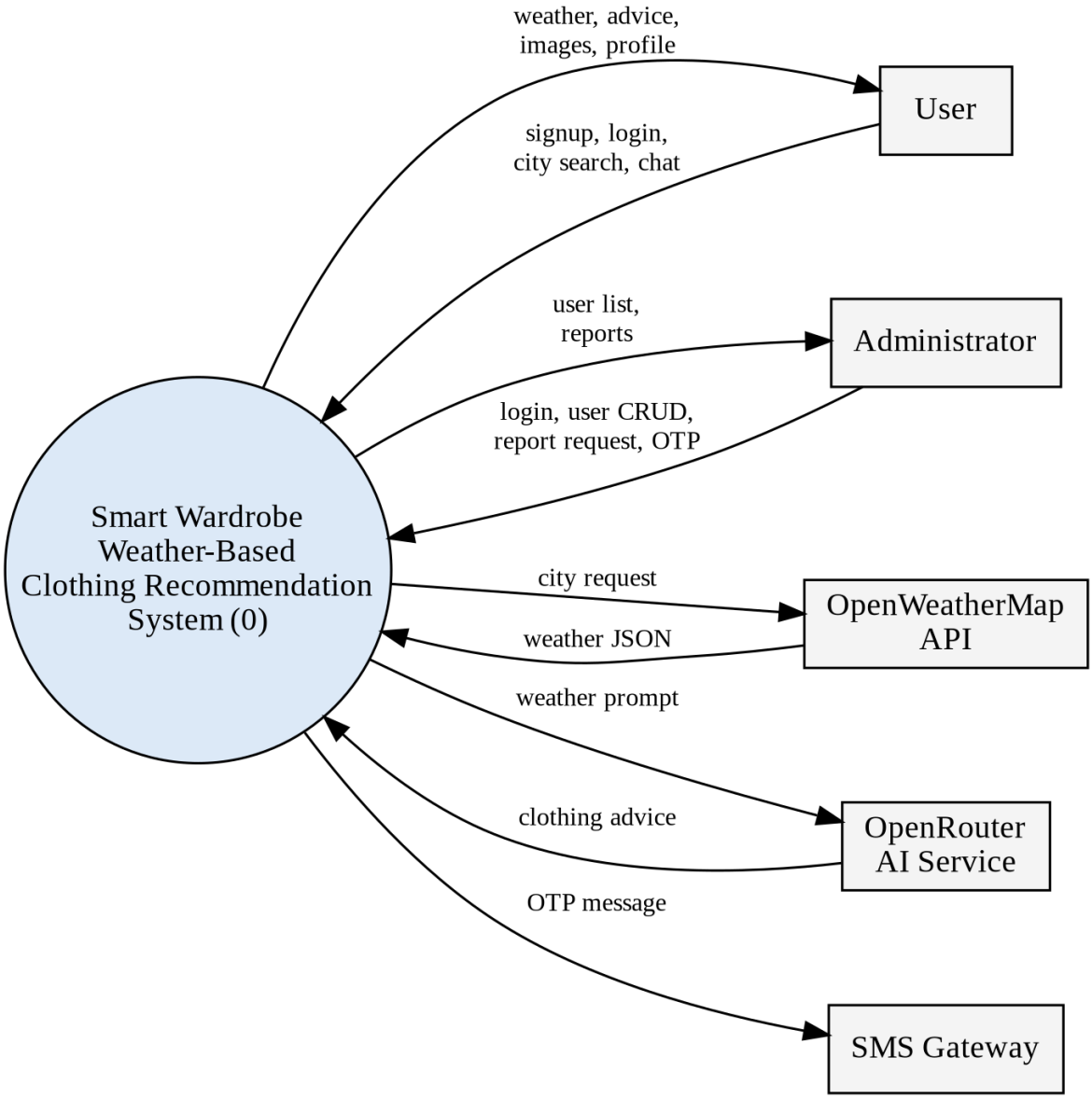


Figure 4.1: Context diagram of Smart Wardrobe

b. Data Flow Diagram (DFD)

The level 1 data flow diagram breaks the system into six processes: authentication, weather retrieval, clothing advice, profile management, admin user management and report generation. D1 and D2 are the users and admin data stores.

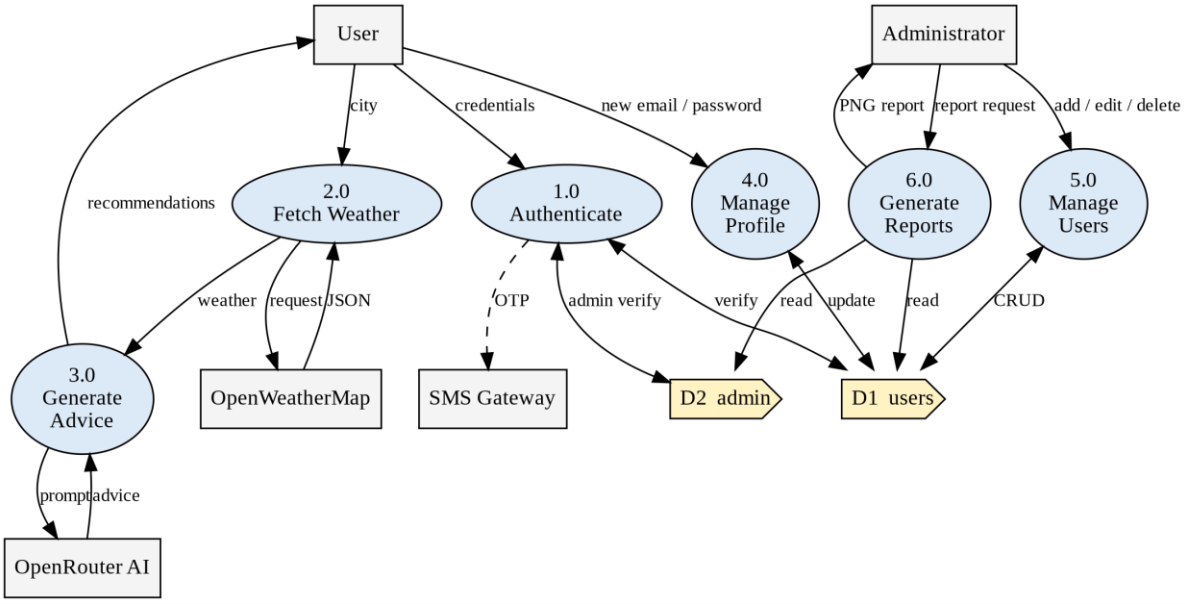


Figure 4.2: Level 1 data flow diagram

c. Entity Relationship Diagram (ERD)

The database has two main tables, users and admin. One administrator manages many users. An earlier prototype also used a third table, logintable, which stored a location with each login; it is not used by the current system and is omitted from the diagram. Weather data and AI recommendations are fetched on demand and are not stored.

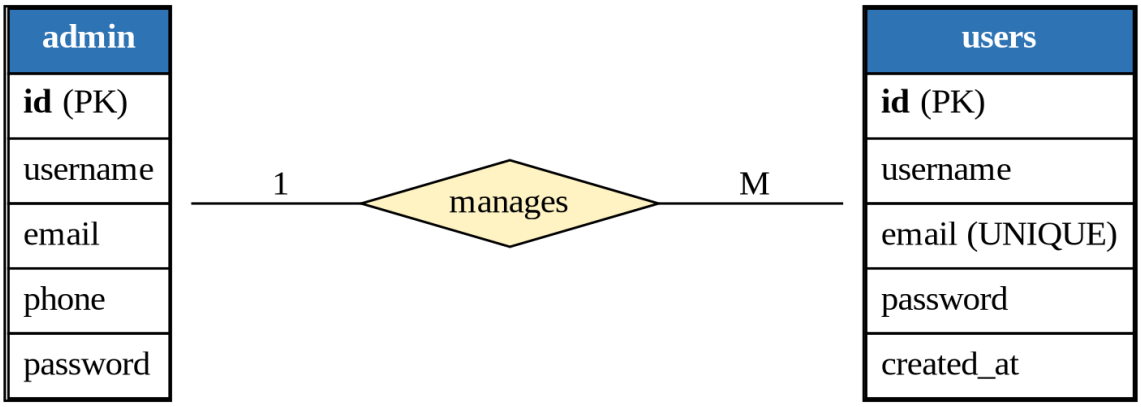


Figure 4.3: Entity relationship diagram

d. Flowchart Diagram

The flowchart shows the main path of a user from the home page to login, weather display, clothing advice and logout.

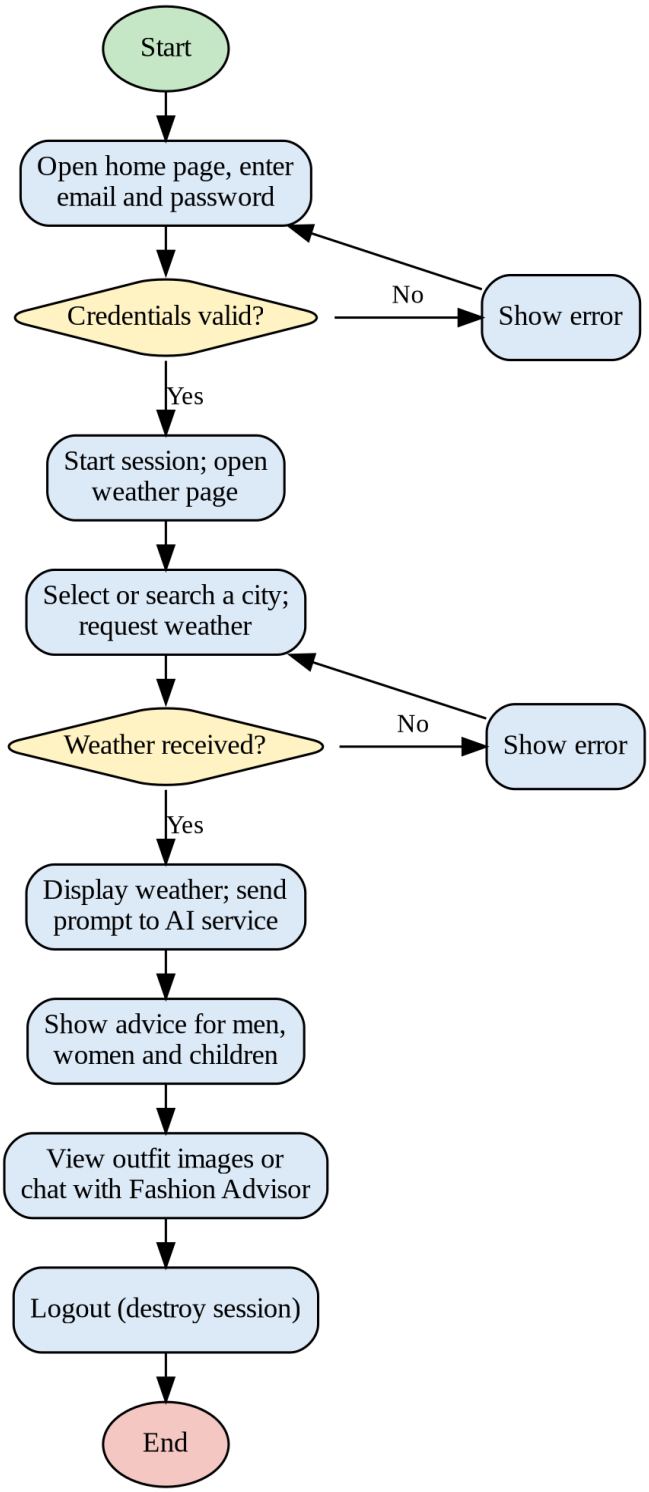


Figure 4.4: Main system flowchart

4.3 System Design

4.3.1 System Architecture

Smart Wardrobe uses a layered, client-server architecture. The browser handles presentation. PHP scripts on an Apache server handle application logic. A MySQL database stores accounts. External APIs supply weather, AI responses and SMS delivery.

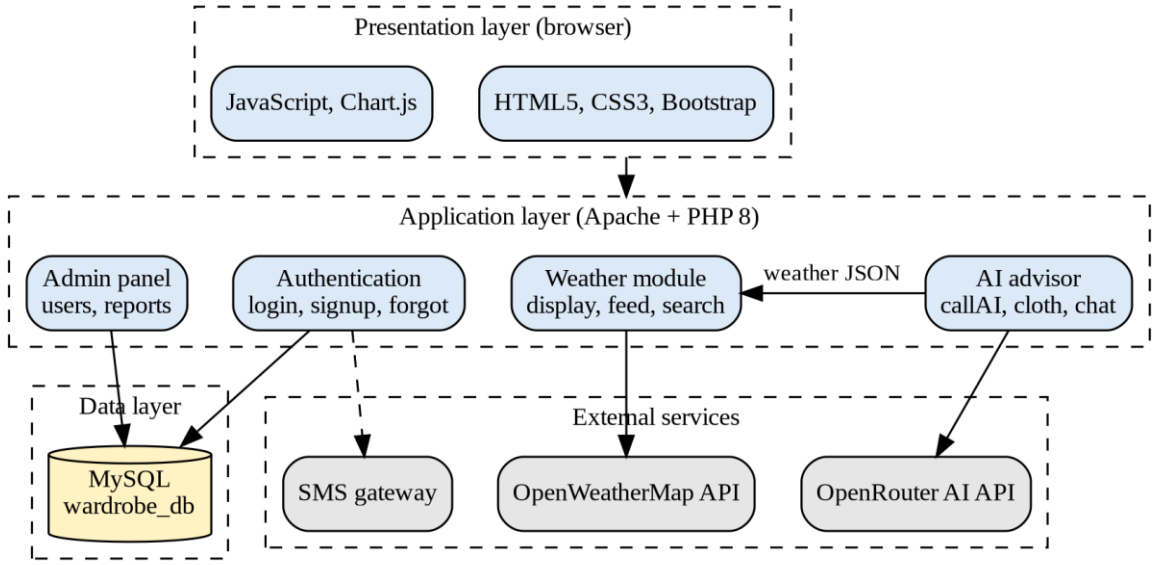


Figure 4.5: System architecture

4.3.2 Database Design

The tables below are the structure expected by the system code. Field sizes are suggested values.

Table: users

Field	Type	Description
id	INT, PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the user
username	VARCHAR(50)	Display name; must be unique
email	VARCHAR(100), UNIQUE	Email address used to log in
password	VARCHAR(255)	User password (see Section 5.0 on hashing)
created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP	Join date used in the users report

Table: admin

Field	Type	Description
id	INT, PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the administrator
username	VARCHAR(50)	Administrator name shown in the sidebar
email	VARCHAR(100)	Email used to log in
phone	VARCHAR(20)	Phone number in +254 format used for OTP delivery
password	VARCHAR(255)	Administrator password

4.3.3 Module Design

Module	Main files	Responsibility
Landing and resources	index.php, blog.html	Welcome page with Login and Signup links; blog and resources
Authentication	signup.php, login.php, logout.php, forgot.php	Register, sign in, sign out and reset password
Weather	kenya_weather.php, weather_display.php, search.php	Fetch, format and display weather; city search
AI advisor	callAI.php, cloth.php, display_callAI.php	Build prompt, call AI, display advice by category
Chat and images	home.php, genimg.php	Fashion Advisor chat; outfit image gallery and download
Profile	profile.php	Change email and password
Database	connection.php	MySQL connection shared by all pages
Administration	login_1.php, admin_dashboard.php, add_user.php, edit_user.php, delete_user.php, profile_1.php, weather_display_1.php	Admin sign-in, user CRUD, profile and weather analysis
Admin recovery and reports	forgot_password.php, generate_report.php	SMS OTP password reset; PNG users report

4.3.4 Interface Design

- **Landing page**: Navigation bar (Home, Login, Signup), welcome message and a "Click Here" button to the resources page.
- **Login and Signup pages**: A centred form box with icons for username, email and password, a show/hide password icon, and links between the two pages.
- **Weather page**: A greeting and navigation tabs (Weather, Cloth, Cloth Adviser, Profile, Logout), a weather card with an icon, temperature and details, and a city search.

- **Clothing recommendations page**: A weather summary followed by three cards for men, women and children, a last-updated time and a Generate Outfit Images button.
- **Fashion Advisor page**: A chat window with message bubbles, timestamps and an input box.
- **Admin pages**: A blue sidebar (Dashboard, Add User, Weather Analysis, Profile, Settings, Report, Logout) and a main content area with a users table or form.

4.4 Implementation

4.4.1 Development Environment

The system was developed on a local XAMPP stack (Apache, MySQL and PHP). Composer was used to manage PHP packages; the project declares the Guzzle HTTP client and PHPMailer in composer.json. The database is named wardrobe_db.

4.4.2 Authentication Module

The signup page validates that all fields are filled and that the email is valid. It uses a prepared statement to check whether the username or email already exists, then inserts the new user and redirects to the login page after three seconds. The login page looks up the user by email, starts a session storing the user id and username, and redirects to the weather page. Private pages begin with a session check and redirect unauthenticated visitors to the login page. The logout page clears session data, expires the session cookie, destroys the session and sends no-cache headers before redirecting to the home page.

4.4.3 Weather Module

Weather is obtained through a WeatherFetcher class that builds the request URL for a city and country code (KE), asks for metric units, sends the request with cURL using a 10-second time-out, and decodes the JSON reply. The values kept are temperature, feels-like temperature, humidity, main condition, description, wind speed, icon code and observation time. The list of cities is held in an array in kenya_weather.php, and a failure for one city does not stop the others.

```
$url = "https://api.openweathermap.org/data/2.5/weather?q="
     . urlencode($city) . "," . urlencode($country)
     . "&units=metric&appid=" . $this->apiKey;
```

The search page lets a user type any city and displays a result card containing the icon, location, temperature, description, feels-like temperature, humidity, wind speed, pressure, visibility and the update time. A loading overlay is shown while the request runs.

4.4.4 AI Recommendation Module

The callAI.php script is the core of the module. It first requests the weather JSON from kenya_weather.php. It then writes a prompt that casts the model as a fashion expert for Kenyan weather, lists each city with its temperature, feels-like temperature, conditions, humidity and wind,

and asks for separate male, female and children recommendations for casual and formal occasions with layering and protection from sun or rain. The prompt is sent to the OpenRouter chat completions endpoint using a free DeepSeek-based model with a limit of 2,000 tokens and a temperature of 0.7. The result is returned as JSON with a status, update time, weather summary and recommendation text.

The cloth.php page calls callAI.php, shows the current weather, separates the response into men's, women's and children's sections, and displays a last-updated time. If the call fails, it shows an error notice together with fallback recommendations so that the page remains useful. The display_callAI.php page presents the same output in a standalone layout.

4.4.5 Fashion Advisor and Outfit Images

The home.php page provides a chat interface. JavaScript sends the user's message to the OpenRouter chat endpoint and appends the reply to the conversation with a timestamp. The genimg.php page builds descriptive prompts for male, female and child outfits, and shows a set of curated images chosen at random for each category. A user can open an image in a modal window and download it. During development, two experimental scripts were also written: open.php calls the Stability AI text-to-image service and saves the picture, and try.php parses the JSON returned by an image service. Both are test utilities and are not linked from the live interface.

4.4.6 Profile Module

The profile page allows a signed-in user to change email or password. In both cases the current password must be entered first. For an email change the system also checks that the new address is not already used by another account. Database access on this page uses prepared statements.

4.4.7 Administrator Module

- **Login**: The admin page checks the email against the admin table (case-insensitive, prepared statement) and opens the dashboard on success.
- **Dashboard**: Lists every user, newest first, with Edit and Delete buttons. Delete asks for confirmation.
- **Add and edit user**: Validates input, hashes the password with password_hash and saves it using prepared statements. When editing, a blank password field keeps the current password.
- **Weather analysis**: Lets the administrator enter any city (default Nairobi) and shows a weather card with a line chart comparing temperature and feels-like temperature, a pie chart of humidity and wind speed, and a Print button.
- **Profile**: Lets the administrator update username, email, phone (normalised to the +254 format) and password.

- **Report**: generate_report.php confirms that the signed-in user is in the admin table, then uses the PHP GD library to draw a PNG listing users with ID, username, email, role (Admin or User) and join date, plus the total number of users.

4.4.8 SMS One-Time Password Reset (Administrator)

The administrator enters a phone number, which is normalised to the international format (for example 0712345678 becomes +254712345678). If the number matches an administrator record, a six-digit random OTP is generated and sent by SMS. The OTP is kept in the session and is valid for five minutes with a maximum of three attempts. When the correct OTP is supplied with a matching new password of at least six characters, the password is hashed and saved.

```
$otp = str_pad(random_int(0, 999999), 6, '0', STR_PAD_LEFT);
$_SESSION['password_reset'] = [ 'phone' => $db_phone,
    'otp' => $otp, 'attempts' => 0, 'expires' => time() + 300 ];
```

4.5 System Testing

Testing was planned at three levels. **Unit testing** checks single pages and functions, such as signup validation. **Integration testing** checks that modules work together, for example weather data flowing into the AI prompt. **System testing** checks complete user journeys from login to logout (Sommerville, 2016). The test cases below define the input and the expected result. The Status column is completed during execution.

ID	Test case	Input / action	Expected result	Status
TC01	Signup with valid details	Username, valid email, password of 6+ characters	Account created; success message; redirect to login	
TC02	Signup with existing email	Email already registered	Message: username or email already exists	
TC03	Signup with invalid email	Text without @	Invalid email error; no account created	
TC04	Login with valid details	Registered email and password	Session started; weather page opens	
TC05	Login with wrong password	Registered email, wrong password	Wrong password message; no session	
TC06	Open private page without login	Open weather_display.php directly	Redirected to login page	
TC07	Weather for default city	Open weather page	Temperature, humidity, wind and conditions shown	
TC08	Search valid city	"Meru"	Result card with weather details	
TC09	Search invalid city	"Xyzabc"	Error card: please try another location	

ID	Test case	Input / action	Expected result	Status
TC10	View AI recommendations	Open cloth.php	Men, women and children sections displayed with update time	
TC11	AI service unavailable	Disconnect internet or use invalid key	Error notice with fallback recommendations	
TC12	Fashion Advisor chat	Ask "What should I wear to a wedding?"	Reply appears in the chat with timestamp	
TC13	Change email in profile	New email and correct current password	Email updated; success message	
TC14	Admin login	Valid admin email and password	Dashboard with users table opens	
TC15	Admin adds user	New username, email, password	User appears in table; success message	
TC16	Admin edits user	Change username; leave password blank	User updated; password unchanged	
TC17	Admin deletes user	Click Delete and confirm	User removed from table	
TC18	Generate report	Open generate_report.php as admin	PNG report with users and total count	
TC19	OTP wrong three times	Enter incorrect OTP three times	Attempts exhausted; OTP session cleared	
TC20	Logout	Click Logout	Session destroyed; redirect to home page	

4.6 Installation and User Guide

4.6.1 Installation Steps

1. Install XAMPP (or any Apache, PHP and MySQL stack) and start the Apache and MySQL services.
2. Copy the project folder into the web root (for example htdocs/smartwardrobe). Place the administrator pages in their own folder, using the file names the pages link to.
3. Create a database named wardrobe_db in phpMyAdmin and create the users and admin tables using the SQL in Appendix B. Insert at least one administrator record.
4. Edit connection.php with the database server, user, password and database name for your environment. Use a dedicated database user with limited rights instead of the root account.
5. Obtain an OpenWeatherMap API key and an OpenRouter API key and place them in the settings used by kenya_weather.php, search.php, weather_display.php and callAI.php.
6. Run "composer install" in the project folder to install the declared libraries.
7. Confirm that the PHP extensions curl, mysqli and gd are enabled in php.ini, then restart Apache.

8. Open http://localhost/smartwardrobe/index.php in a browser to start using the system.

4.6.2 User Guide

For Users

1. Open the home page and choose **Signup**. Enter a username, email and password, then submit.
2. Log in with your email and password. The weather page opens.
3. Use the **Search** box to look up any city, for example "Mombasa".
4. Open the **Cloth** tab to see clothing advice for men, women and children, and press **Generate Outfit Images** for visual ideas.
5. Open **Cloth Adviser** to chat with the Fashion Advisor.
6. Open **Profile** to change your email or password, and use **Logout** when you finish.

For Administrators

1. Open the admin login page and sign in with the administrator email and password.
2. On the **Dashboard**, review the list of users. Use **Add User** to create an account, and **Edit** or **Delete** beside any user to change or remove it.
3. Open **Weather Analysis** to view charts for a city and press **Print Analysis** if a copy is needed.
4. Open **Report** to generate the users report image.
5. If the password is forgotten, choose **Forgot password**, enter the registered phone number, type the OTP received by SMS and set a new password.

CHAPTER 5: LIMITATIONS, RECOMMENDATIONS AND CONCLUSION

5.0 Challenges and Limitations of the Current Build

Reviewing the source code against the objectives shows that the system meets its main functional goals, but a number of limitations remain. They are recorded here so that they can be corrected in the next version.

Security-related limitations

- **Password handling is inconsistent.** Signup, login, the forgot password page and the profile page for ordinary users store and compare passwords as plain text, while the administrator tools (add user, edit user, SMS reset) hash passwords with password_hash. As a result, an account created or edited by an administrator has a hashed password that the plain-text login page cannot verify, and an administrator who resets a password by SMS can no longer log in with the plain-text admin login.
- **Some queries are built from raw input.** The user login and forgot password pages and the administrator profile page place input directly into SQL text. Prepared statements, already used on other pages, should be used everywhere to prevent SQL injection.
- **Secrets are stored in source files.** Database credentials and API keys appear directly in several PHP files, and the database connection uses the root account.
- **Password reset for users is not verified.** The user forgot password page changes a password knowing only the email address.
- **Access control on admin pages is incomplete.** Several admin pages only check that a session exists. A role check against the admin table should be added so that ordinary users cannot open them. Deletion is also triggered by a simple link and should use a POST request with a token.
- **Output is not always escaped.** Some messages and error text are printed without htmlspecialchars.

Functional limitations

- **Single default city.** The recommendation module currently requests weather for Nairobi only, although the code supports a list of eleven cities.
- **No stored history.** Weather readings, recommendations and chat messages are not saved, so users cannot review earlier advice.
- **Dependence on external services.** Weather, AI and SMS features need internet access and valid API keys, and free AI models may be slow or rate limited.
- **Outfit images are curated, not generated.** Images are chosen from a fixed list and are not matched exactly to the AI text. The experimental image generation scripts are not part of the live interface.
- **Settings page.** The administrator sidebar links to a settings page that has not yet been built.

- **Duplicate files.** Some files appear in user and administrator versions with similar names, which makes deployment and maintenance harder.

5.1 Recommendations

1. **Hash all passwords** with password_hash and verify them with password_verify on every login page, and migrate existing plain-text passwords by forcing a reset.
2. **Use prepared statements everywhere** for database access and validate all input on the server.
3. **Move secrets out of source code.** Keep API keys and database credentials in environment variables or a configuration file outside the public web folder, create a limited database user, and rotate any key that has been shared or stored in a public place.
4. **Add role-based access control** by checking the admin table on every admin page, and protect delete and update actions with POST requests and CSRF tokens.
5. **Secure the user password reset** by emailing a time-limited token or sending an OTP, as already done for administrators.
6. **Enable HTTPS** on the production server and set secure, HttpOnly and SameSite cookie flags.
7. **Support more cities and user preferences** so that each user can save a home city and style preferences.
8. **Cache weather and AI responses** for a short period to reduce API calls, speed up pages and stay within free limits.
9. **Store history** of recommendations and favourite outfits in the database.
10. **Match outfit images to advice** by using an image generation service that follows the recommendation text, with moderation and caching.
11. **Improve error handling and logging** so that failures are written to a log file and users see friendly messages.
12. **Complete the settings page** and merge duplicate user and administrator files into a single structure with shared includes.
13. **Test on real users** and update the Status column of the test plan, then act on the feedback collected.
14. **Comply with the Data Protection Act, 2019** by adding a privacy notice and consent text on the signup page and by allowing users to request deletion of their data.

5.2 Future Work

- A mobile application for Android and iOS with push notifications such as "Take an umbrella today".
- A personal digital wardrobe where users upload photographs of their own clothes and receive outfit combinations from items they already own.

- Multi-day forecasts and event-based planning for trips, weddings and interviews.
- Swahili language support for the interface and the Fashion Advisor.
- Links to local clothing sellers for items recommended by the system.

5.3 Conclusion

Smart Wardrobe set out to help people in Kenya decide what to wear by linking live weather data to clear clothing advice. The project delivered a working web application that retrieves real-time weather from OpenWeatherMap, builds structured prompts for an AI model through OpenRouter, and displays recommendations for men, women and children. It also provides a Fashion Advisor chat, outfit images, user accounts and profiles, and an administrator panel with user management, weather analysis, SMS-based password recovery and a users report.

The development followed an Agile approach and produced analysis and design models including a context diagram, data flow diagram, entity relationship diagram, flowchart and architecture. The review of the code also identified limitations, mainly in password handling, query safety and access control, together with recommendations to correct them. With these improvements, and with features such as saved preferences, multi-day forecasts and a mobile application, Smart Wardrobe can grow into a dependable everyday tool for Kenyan users.

5.4 References

1. ASHRAE. (2020). *ANSI/ASHRAE Standard 55-2020: Thermal environmental conditions for human occupancy*. American Society of Heating, Refrigerating and Air-Conditioning Engineers.
2. Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
3. Fanger, P. O. (1970). *Thermal comfort: Analysis and applications in environmental engineering*. Danish Technical Press.
4. Government of Kenya. (2019). *The Data Protection Act, 2019*. National Council for Law Reporting.
5. OpenRouter. (n.d.). *OpenRouter API documentation*. https://openrouter.ai/docs
6. OpenWeather. (n.d.). *Current weather data API documentation*. https://openweathermap.org/current
7. OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/
8. Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). *Recommender systems handbook* (2nd ed.). Springer.
9. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 10684-10695.

10. Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org
11. Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
12. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

APPENDIX A: SOURCE FILE INVENTORY

The table lists the files that make up the Smart Wardrobe project. Files with a _1 suffix are the administrator versions of pages that also exist for ordinary users.

File	Group	Description
index.php	Landing	Home page with navigation to Login, Signup and resources
home_1.php	Landing	Alternative home page with Login link only
blog.html	Resources	Blog and resources: fashion tips, sustainability, event guides
signup.php	Authentication	User registration with validation and duplicate check
login.php	Authentication	User login and session start
logout.php	Authentication	Secure session destruction and redirect
forgot.php	Authentication	User password reset by email
connection.php	Database	MySQL connection (wardrobe_db)
kenya_weather.php	Weather	JSON weather feed for the configured city list
weather_display.php	Weather	User weather dashboard with icons
search.php	Weather	Search weather of any city
callAI.php	AI	Builds prompt and calls AI service; returns JSON
cloth.php	AI	Clothing recommendations by gender with outfit image button
display_callAI.php	AI	Standalone display of AI recommendations
home.php / home.html	AI	Fashion Advisor chatbot
genimg.php	AI	Outfit image gallery, detail view and download
open.php	Experimental	Test of Stability AI text-to-image generation
try.php	Experimental	Test of parsing an image service JSON response
profile.php	Profile	User email and password update
wardrobelogin.php	Prototype	Earlier login and signup prototype using the logintable table
login_1.php	Admin	Administrator login
logout_1.php	Admin	Administrator logout
admin_dashboard.php	Admin	Users table with edit and delete actions
add_user.php	Admin	Create a user account
edit_user.php	Admin	Edit a user account
delete_user.php	Admin	Delete a user account
profile_1.php	Admin	Administrator profile update

File	Group	Description
forgot_password.php	Admin	SMS one-time password reset
weather_display_1.php	Admin	Weather analysis with charts and print
kenya_weather_1.php	Admin	Weather feed for eleven Kenyan cities
generate_report.php	Admin	PNG users report using GD
connection_1.php	Admin	Database connection for admin pages
composer.json, composer.lock	Dependencies	Guzzle and PHPMailer packages
1mg1.png, generated_image_*.png	Images	Sample wardrobe image and a test generated image

Note: vibelogin.php (a Wi-Fi hotspot login template) was included in the uploaded folder but is not part of the Smart Wardrobe system and is therefore not documented.

APPENDIX B: DATABASE SCRIPT

The script below was reconstructed from the queries used in the source code. Adjust field sizes if needed.

```
CREATE DATABASE IF NOT EXISTS wardrobe_db;
USE wardrobe_db;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE admin (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  phone VARCHAR(20),
  password VARCHAR(255) NOT NULL
);
```

After creating the tables, insert an administrator row using a password hash generated with PHP's password_hash function, and make sure the admin login page verifies passwords with password_verify (see Section 5.0).